Advanced Engineering Informatics 42 (2019) 100935

Contents lists available at ScienceDirect

Advanced Engineering Informatics

journal homepage: www.elsevier.com/locate/aei

Full length article

Deep topology network: A framework based on feedback adjustment learning rate for image classification

Long Fan, Tao Zhang*, Xin Zhao*, Hao Wang, Mingming Zheng

School of Electrical and Information Engineering, Tianjin University, 300072 Tianjin, China

ARTICLE INFO

Keywords:
Deep topology network
Convolutional neural network
Feedback adjustment learning rate

ABSTRACT

Convolutional Neural Network (CNN) has demonstrated its superior ability to achieve amazing accuracy in computer vision field. However, due to the limitation of network depth and computational complexity, it is still difficult to obtain the best classification results for the specific image classification tasks. In order to improve classification performance without increasing network depth, a new Deep Topology Network (DTN) framework is proposed. The key idea of DTN is based on the iteration of multiple learning rate feedback. The framework consists of multiple sub-networks and each sub-network has its own learning rate. After the determined iteration period, these learning rates can be adjusted according to the feedback of training accuracy, in the feature learning process, the optimal learning rate is updated iteratively to optimize the loss function. In practice, the proposed DTN framework is applied to several state-of-the-art deep networks, and its performance is tested by extensive experiments and comprehensive evaluations of CIFAR-10 and MNIST benchmarks. Experimental results show that most deep networks can benefit from the DTN framework with an accuracy of 99.5% on MINIST dataset, which is 5.9% higher than that on the CIFAR-10 benchmark.

1. Introduction

Current popular algorithms primarily use Convolutional Neural Network (CNN) to perform feature extraction and classification. CNN is gradually entering a number of machine learning application domains and is currently making drastically contribution to the state-of-the-art field of computer vision, including tasks such as object detection [1–3], classification [4,5], 3D object recognition [6], and segmentation [7]. CNN is also widely used in various fields such as image/speech classification [8,9], speech recognition and remote sensing image processing [10,11]. Unlike traditional method, CNN model is an end-to-end network architecture that integrates automatic feature extraction and classification. CNN has achieved remarkable results in the field of image processing.

The depth of CNN is an important factor that affects network classification performance. In the 1990s, LeCun et al. [12,13] designed a multi-layered artificial neural network to classify handwritten numbers, named LeNet-5. In the task of the handwritten numbers classification, LeNet-5 achieve a high classification accuracy, but it is processing ability of complex problems was not satisfactory. After that, many methods have been devised aiming for improving the performance of CNN. Among them, Krizhevsky et al. [14] proposed AlexNet structural framework, and made a predominant breakthrough in image recognition tasks. Hinton et al. [15] used nonlinear activation functions and obtained excellent results. In later work, researchers proposed some other related algorithms and acquired improvements accordingly. The most famous are ZFNet [16], VggNet [17] and ResNet [18].

By increasing the depth, the network can use the increased nonlinearity to derive an approximate structure of the objective function and yield a better characterization. However, this also increases the overall complexity of the network and may lead to overfitting. Meanwhile, since the fully connected layers in the CNN form a classical neural network classifier, which is trained by a conventional gradient descent-based implementation, the generalization ability is limited [19]. Some details about generalization ability that affects the network are as follows: First, it is inseparable from sample characteristics. When the training samples are sufficient to characterize some key features or essential features of the research problem, a reasonable network learning mechanism can make it a certain generalization. Second, it is the factor of the network itself. It is worth mentioning that the proposed framework in the paper has strong generalization ability compared to other networks with the same type. A DTN framework is proposed to improve network performance by changing the width of the neural network.

In addition to increasing the depth of the network, other strategies have been used to achieve higher classification performance. On one

* Corresponding authors.
E-mail addresses: zhangtao@tju.edu.cn (T. Zhang), zhaoxin_16@tju.edu.cn (X. Zhao).

https://doi.org/10.1016/j.aei.2019.100935
Received 8 January 2019; Received in revised form 26 May 2019; Accepted 1 June 2019
Available online 20 June 2019
1474-0346/ © 2019 Elsevier Ltd. All rights reserved.

hand, Refs. [20–22] used sampling algorithm to change the distribution of sample classes and then to improve the generalization ability of the model. Besides, other similar algorithms, for example Synthetic Minority Oversampling Technique (SMOTE) [23], which are interpolations of intra-class neighboring samples, to augment the minority class. It cannot effectively generalize sample classification. EasyEnsemble [24] samples several subsets from the majority class, trains a learner using each of them, and combines the outputs of those learners. Khan et al. [25] proposed a cost-sensitive deep CNN to deal with the class-imbalance problem. This method automatically learn feature representations for both the majority and minority classes. Above these methods mainly preprocess the training data of the network to improve network performance. Therefore, the robustness of these methods is directly affected by the training data and have certain limitations and instability in practical applications [26]. On the other hand, learning rate, activation function [39], network structure, batch normalization [27–29] and other parameters of CNN can also improve performance. Among them, Leslie [30] proposed a method of training neural networks using cyclic learning rates. This method allowed the learning rate to cycle through reasonable boundary values and achieved higher classification performance.

Based on the above analysis, this paper focuses on improving the classification performance without increasing the depth of the network. A DTN framework is proposed based on feedback adjustment learning rate. The network consists of multiple topology nodes and each node is a basic network structure model. During the training process, the nodes are assigned with different learning rates which can be adjusted continuously. The adjustment is performed at the beginning of each new training period, a learning rate seed is first selected from the current learning rate set based on the feedback of the training accuracy. Then a new learning rate set is generated by the seed and been used to update the learning rates. The salient contribution of this paper can be summarized as follows:

(1) Multiple sub-networks are applied to the CNN structure as basic network nodes to create a common DTN framework. In the case of maintaining a certain network depth, the horizontal structure of the network is greatly enriched, and the network obtains stronger generalization performance.
(2) Before summarizing the advantages and disadvantages of the existing network structure, the affect of the network structure on the learning rate and the data promotion on the generalization ability is analyzed.
(3) A feedback adjustment learning rate strategy is proposed, which updates the learning rate by using the accuracy of the current network as a feedback factor.

The rest of this article is organized as follows. Section 2 briefly reviews deep neural networks from three different perspectives: depth, structure and learning rate. Section 3 details the proposed DTN framework and the learning rate update algorithm of the network. Section 4 evaluates the proposed DTN framework on two datasets with accuracy. Finally, Section 5 makes conclusions and conducts discussions for future work.

2. Related work

In this section, we will further introduce some related work which adopted strategies rather than increasing network depth to improve the performance of CNN. These work achieved higher performance from two aspects: improving the network structure and adjusting the learning rate.

2.1. Improving the network structure

In order to improve the classification and generalization performance of CNNs, Li et al. [31] proposed a new Depth Variance Network (DVN) that incorporated a hierarchical Bayesian model into a powerful CNN framework. The framework was able to transfer iterative feature distributions from the complete training dataset of one object to the incomplete training dataset of other objects. Szegedy et al. [32] broke the conventional convolutional layer series mode and proposed a method of parallel combination of convolutional layers, which combined 1×1, 3×3, 5×5 convolutional layers and 3×3 pooling layers into an inception. Since the number of parameters of the fully connected layer is fixed, He et al. [33] proposed a new pooling method called spatial pyramid pooling. These methods did not increase the number of network layers, they improved the performance of CNN by balancing training datasets and expanding network width.

2.2. Adjusting the learning rate

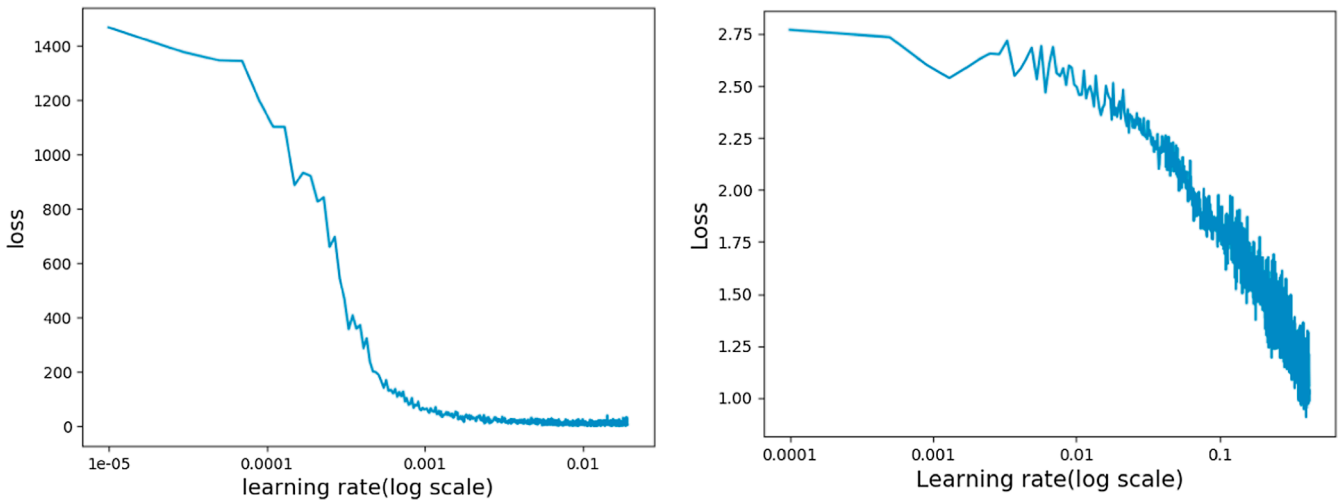
It is well known that learning rate is an important hyper-parameter in CNN. In the previous studies, people set the start learning rate according to the experience, and used the method of piecewise constant attenuation, exponential decay, natural exponential decay and some others to update the learning rate. Leslie [30] proposed a new method called Cyclic Learning Rate (CLR), which adjusted the learning rate cyclically within a reasonable interval. It was concluded that the learning rate which brought the fastest change of the loss function was the optimal. Leslie [30] also proposed a simple way to estimate "reasonable bounds" – linearly increasing the learning rate of the network for a few periods. Based on [30], the interval of the best learning rate is analyzed. First, a very small start learning rate is set, and then the loss of each batch is calculated after improving the learning rate. Finally, the curve of learning rate and loss is drawn. Fig. 1(a) shows the loss functions of the MNIST dataset. As the learning rate increases, the loss of the network gradually decreases and then increases. In the experiment, the start learning rate is set to $5e-5$, and the step size is set to $1e-5$. As shown in Fig. 1(a), the loss function has the largest slope within $[1e-4, 1e-3]$, so the start learning rate is set to $5e-4$. Fig. 1(b) shows the curve of the loss function and the learning rate on CIFAR-10 dataset. The start learning rate and the step size are set to $le-4$. According to Fig. 1(b), it can be concluded that the optimal learning rate ranges from $1e-4$ to $1e-3$.

3. DTN architecture

As shown in Fig. 2, it is a multi-learning rate Topological Neural Network (DTN) framework based on CNN. The key technical elements of DTN are outlined briefly as follows. First, a DTN framework is defined based on feedback adjustment learning rate. In this framework, the traditional CNN structure is used as a topology node, which constructs a hyper-network with the same sub-network and different learning rates. Second, a predictive probability fusion module is defined, and generates network prediction values by combining the predicted probabilities of each node output. Third, during the training process, we introduced an optimization algorithm to adjust the learning rates based on the feedback of the accuracy.

3.1. Hyper-network structure based on CNN

The core of CNN lies in feature learning. The acquisition of its hierarchical feature relies on hierarchical network, based on which the problem of traditional manual feature extraction can be solved. The current deep network mainly improves the network performance by expanding the single network structure. For instance, VggNet [17] and ResNet [18] increase the depth of the network while GoogleNet [32] increases the width of the network by using convolution kernels of different sizes for each layer. Considering different capabilities in feature extraction and decision-making in different network structures, a DTN framework is proposed. The topology neural network is composed



(a) MNIST (b) CIFAR-10

Fig. 1. The curve of loss function of learning rate.

of multi-node neural networks, including network discriminant nodes, discriminant decision nodes and learning rate update nodes. The number of nodes in hyper-network is denoted by N. The network node set is defined as follows:

$$\phi_N = \{\varphi_1, \cdots, \varphi_i, \Psi_N, G_N\} \tag{1}$$

where φ_i is the discriminant node of the topology network, Ψ_N is the decision node, and G_N is the learning rate update node. Each discriminant node φ_i has a certain discriminative decision-making ability. The type of the input and the prediction probability p_i are determined by the feature extracted by the current network.

The prediction probability set is defined as $P_N = \{p_1, \cdots, p_i\}$. The discriminant node is defined as $\varphi_i = (x, \gamma_i)$, where x denotes the dataset used by current network for training, and γ_i denotes the learning rate used by the i-th node. To comprehensively evaluate the performance of each discriminant node, the discriminant decision function is defined as:

$$\Psi_N(P) = \mathrm{softmax}(p_1, \cdots, p_i) \tag{2}$$

where p_i denotes the prediction probability of the network discriminant node φ_i. The discriminant decision node Ψ_N makes a discriminant decision by the predicted value p_i obtained from all discriminant nodes φ_i. The update of learning rate has a great affect on the training performance of the network, so multiple learning rates are applied in the

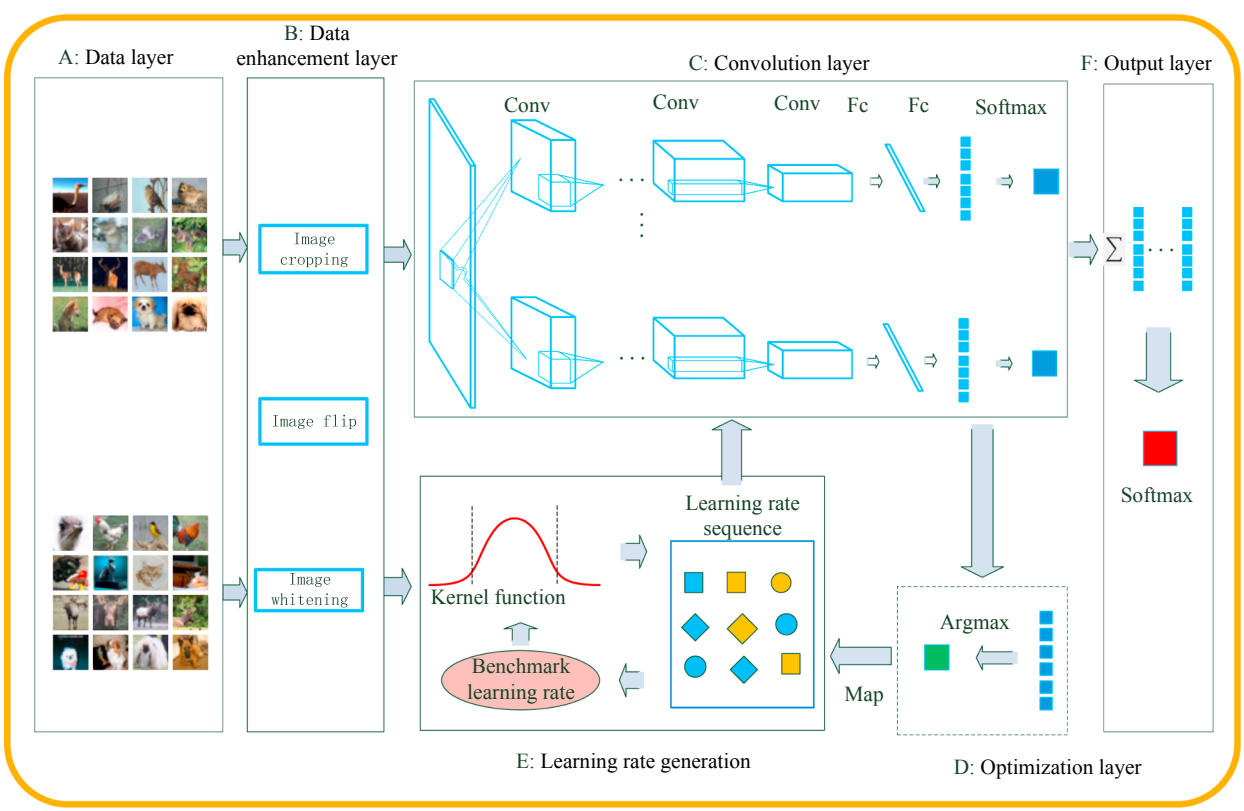


Fig. 2. The pipeline of proposed DTN framework.

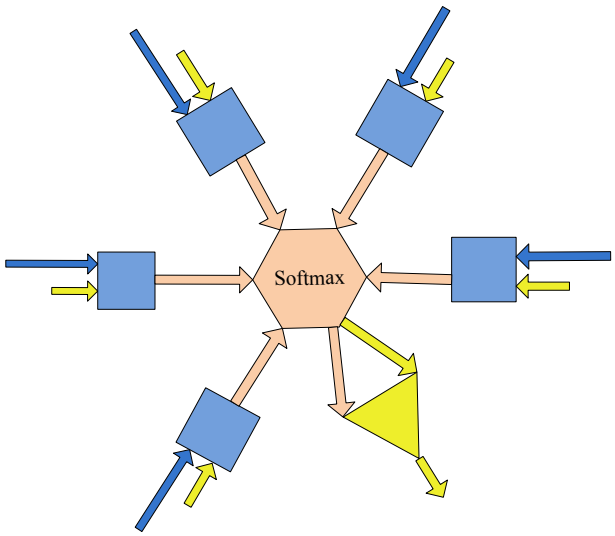


Fig. 3. Star topology connection. *Notes:* Blue square denotes sub-network node, orange polygon denotes probability fusion node, yellow triangle denotes learning rate update node. Blue arrow denotes data flow direction and yellow arrow denotes learning rate flow direction. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

training process. The learning rate of each node can be updated by uniformly distributed linear function or Gaussian distributed nonlinear function. The relationship between learning rate and network loss shows a normal distribution. Thus, we introduce a Gaussian function. The update function of learning rate is defined as follows:

$$G_N(\gamma) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(\gamma_b-\mu)^2}{2\sigma^2}} \tag{3}$$

where $G_N(\gamma)$ denotes the generated learning rate sequence, γ_b denotes the starting learning rate, μ denotes the mean value of the learning rate update coefficient in its variation range, and σ denotes the length of the variation range. The mapping relation between the learning rate and the prediction probability is denoted as $\mathscr{M}_{N=}(V_N, P_N)$, where $V_N = \{\gamma_1, \cdots, \gamma_i\}$ denotes a set of learning rates. As shown in Fig. 3, this is a star topology connected neural network. Each blue square node represents a single sub-network structure in Fig. 2(C). The decision node Ψ_N (as shown in Fig. 2(D)) firstly fuses the probability of each node and then makes a discriminating decision through the *Softmax* function. The learning rate generation node G_N (as shown in Fig. 2(E)) is used to generate a learning rate sequence V_N, which is suitable for all discriminant nodes φ_i.

3.2. Learning rate update

Deep learning models are usually trained by gradient descent algorithms or their improved algorithms. A common feature of this algorithm is to initialize a target solution, and then determine the search direction and the step size according to which the initial solution shift. If the prediction error of the neural network decreases, the new solutions can be updated repeatedly to obtain the minimum network prediction error. The step size in the gradient descent method is the product of the learning rate and the gradient. In the back propagation, the update the learning rate formula is defined as follows:

$$w_n \leftarrow w_n - \eta \frac{\partial L}{\partial w_n} \tag{4}$$

where the learning rate η is used to control the speed. On high learning rate, the change of network weights can be dramatic, leading the optimal value being skipped. Gradient disappearance and explosion problems may therefore easily occur. When the learning rate is reduced, the step size in the direction of the minimum value of the loss function will become smaller, which makes the training more reliable.

An optimal iterative learning rate algorithm is proposed. The learning rate seed is defined as γ_b and the learning rate updated period is defined as t, which dynamically adjusts the learning rate. Besides, the learning rate update network Θ_N is embedded into hyper-network ϕ_N. The partial verification dataset is used to verify the detection rate of networks with different learning strategies, and obtain the prediction rate $p_i^{(t)}$ of each sub-network. The predicted probability values of various categories are utilized to make discrimination decision node Ψ_N as shown in follows:

$$\Psi_N.\ p^{(t)} = \Psi_N(p_1^{(t)}, p_2^{(t)} \cdots, p_n^{(t)}),\ p_i^{(t)} \in P \tag{5}$$

where $p^{(t)}$ denotes the optimal predicted value of the *t*-th learning rate period. Learning rate update function is defined as follows:

$$Lr^{t+1} = Lr^{(t)} \cdot h(\delta_p) = \mathscr{M}_N(\gamma_b, p^{(t)}) \cdot h(\delta_p{}^{(t)}) \tag{6}$$

where $\delta_p{}^{(t)} = p^{(t)} - p^{(t-1)}$ denotes the change of predicted rate, $\mathscr{M}_{N=}(V_N, P_N)$ denotes the mapping relationship between learning rate set and prediction set. The detailed algorithm is showed in **Algorithm 1**.

In order to effectively implement the feedback mechanism to optimize the learning rate, the threshold function is defined as the follows:

$$h(x) = \begin{cases} 1 & x > \theta \\ 0 & \text{others} \end{cases} \tag{7}$$

where θ denotes the update threshold.

In Ref. [34], the authors studied how the specific choice of the loss function affects the depth model and its learning dynamics, and the resulting classifier's robustness to various effects. Through experimental

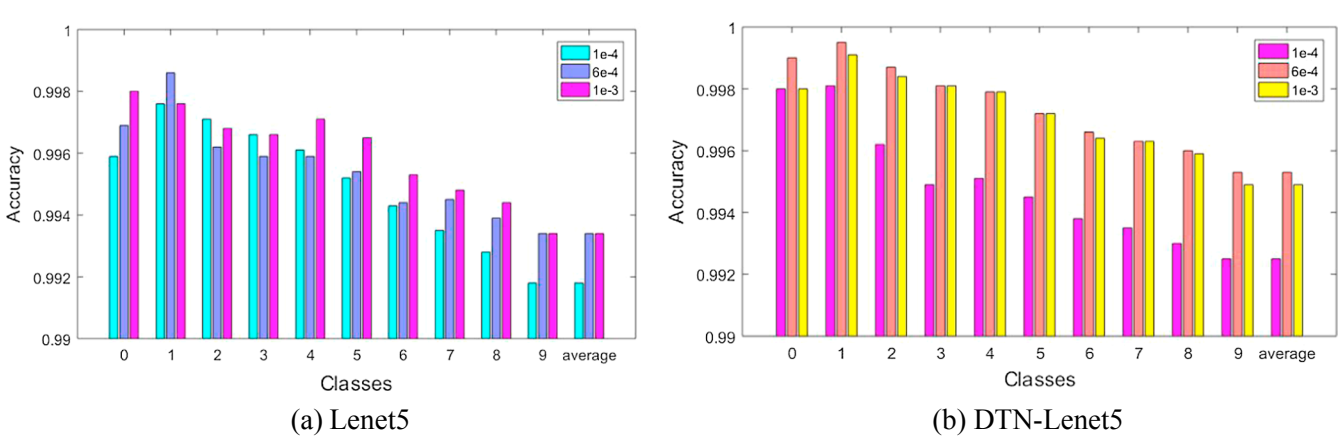
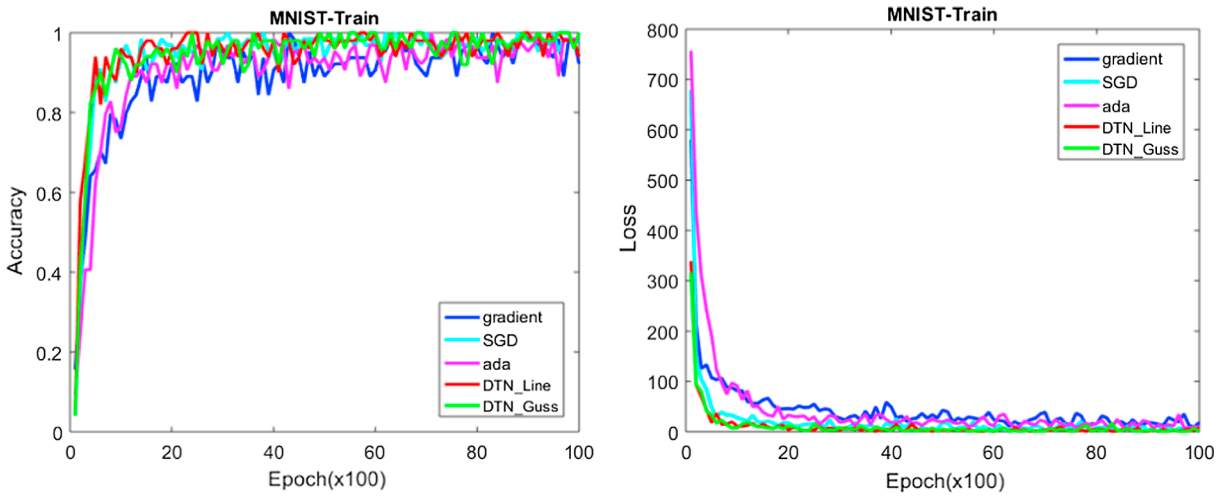


Fig. 4. The comparison of accuracy between Lenet5 and DTN-Lenet5 on MNIST.



(a) The curve of accuracy rate (b) The curve of loss

Fig. 5. The curve comparison of different methods on MNIST dataset.

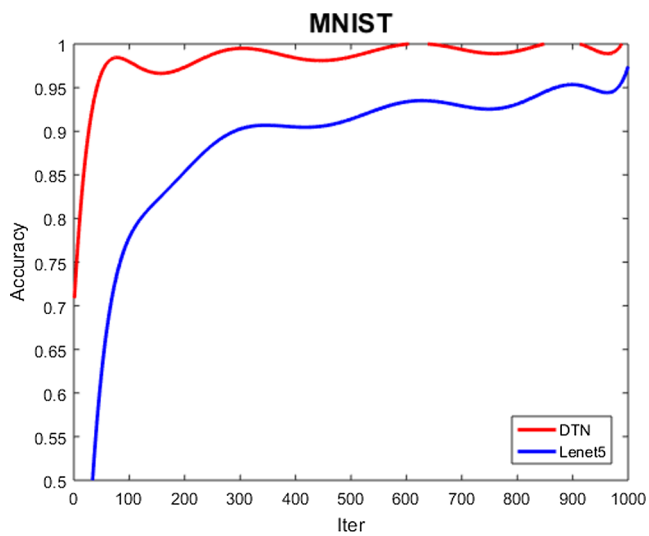


Fig. 6. The fitting curve of the first 1000 iterations.

Table 1
The performance comparison of Lenet5 and DTN-Lenet5 on MNIST testing dataset with different update methods.

ID	Model	Update method	Accuracy
1	Lenet5 [13]	–	0.9910
2	Lenet5	Adam	0.9931
3	Lenet5	Gradient	0.9905
4	Lenet5	Momentum	0.9930
5	DTN-Lenet5	Line_gd	0.9943
6	DTN-Lenet5	Guss_gd	0.9940
7	DTN-Lenet5	Line_mom	0.9934
8	DTN-Lenet5	Guss_mom	**0.9953 (highest accuracy)**

comparison, the cross-entropy loss function shows significant advantages. Therefore, the cross-entropy loss function is chosen in the experiment. The cross-entropy cost function is defined as follows:

$$\text{Loss} = -\frac{1}{n}\sum\left[y\ln\alpha + (1-y)\ln(1-\alpha)\right] \tag{8}$$

where $\alpha = \sigma(\sum w_j x_j + b)$ denotes node internal neuron output. The gradient descent method is used to optimize the loss function of each node under different learning strategies.

Algorithm 1 DTN training process.

1: **Input**: Original Training Dataset: x
2: **Output**: Classification Probability: pred
3: Initialize learning rate seed: γ_b
4: Initialize the generation function mean: μ and standard deviation: σ
5: **while** Loss term not converged **do**
6:**for** t∈ [1, 2, ⋯t_max] **do**
7: Randomly generate N learning rates:
8: $\gamma_1^t, \gamma_2^t, \cdots\gamma_N^t \leftarrow \frac{1}{\sigma\sqrt{2\pi}}e^{-\frac{(\gamma_b-\mu)^2}{2\sigma^2}}$
9:**for** i∈ [1, 2, ⋯m] **do**
10:Sample $\{x_i\}_{i=1}^m$ P a mini-batch from training dataset
11: $\varphi_1^{(t)}(x), \varphi_2^{(t)}(x), ..\varphi_N^{(t)}(x) \leftarrow$ CNN forward and back propagations
12: $w_n^1 \leftarrow w_n^1 - \gamma_1^t \frac{\partial L^1}{\partial w_n^1}$
13: $w_n^2 \leftarrow w_n^2 - \gamma_2^t \frac{\partial L^2}{\partial w_n^2}$
14:…
15: $w_n^N \leftarrow w_n^N - \gamma_N^t \frac{\partial L^N}{\partial w_n^N}$
16:**end for**
17:Sample $\{x_i\}_{i=1}^m$ P a mini-batch from verification dataset
18: $p_1^{(t)}, p_2^{(t)}\cdots, p_N^{(t)} \leftarrow \phi_N(x)$
19: $p^{(t)} = max(p_1^{(t)}, p_2^{(t)}\cdots, p_N^{(t)})$
20: $\delta_p{}^{(t)} = p^{(t)} - p^{(t-1)}$
21:**if** $\delta_p{}^{(t)} >$ Threshold **do**
22:The mapping relationship between learning rate and prediction rate is $\mathscr{M}_{N=}(V_N, P_N)$
23: $Lr^{(t)} \leftarrow \gamma_b \leftarrow \mathscr{M}_N(p^{(t)})$
24:**end if**
25: $Lr^{t+1} \leftarrow Lr^{(t)}$
26: pred ← Softmax$(\sum_{i=1}^N p_i^{(t)}/N)$
27:**end for**
28: **end while**

4. Experiments and discussion

4.1. Experimental settings

The proposed DTN framework is implemented on 2 GPU clusters using *TensorFlow* with 32 NVIDIA Tesla V100-PCIE-16GB. A multi-learning rate model is trained to transfer the latent variable Lr, which denotes current optimal learning rate. After that, the learning rates generated by current iteration are assigned to each node. The training process will be stopped when the loss function keeps stable. Further, to

Table 2
The performance comparison of DTN-Lenet5 and other networks on MNIST testing dataset.

ID	Dataset	Network	Accuracy
	MNIST	VggNet [17]	0.989
	MNIST	GoogleNet [32]	0.992
	MNIST	Network in network [37]	0.989
	MNIST	Simple network	0.974
	MNIST	DTN_Guss	**0.995 (highest accuracy)**

Table 3
The computation time comparison of DTN-Lenet5 and Lenet5 on MNIST testing dataset.

Model	Iterations	Training time (s)	Testing time (s)	Accuracy
Lenet5	20,000	76	0.44	0.9880
Lenet5	50,000	136	0.40	0.9905
DTN-Lenet5	20,000	223	0.76	0.9943

verify the effectiveness of the proposed DTN framework, it is applied to equip three types of state-of-the-art CNN, and conduct extensive experiments over two public benchmarks: MNIST [35] and CIFAR-10. The first type is the deep residual network [18], which has a good effect on the network structure and generalization performance. The second type Lenet-5. It is a shallow structure network and can achieve extraordinary accuracy on certain datasets. The third type is the VggNet [32] with a medium depth structure. All networks involved are trained in two ways: with DTN and without DTN. As for the evaluation, these methods are implemented according to the experimental settings documented in their original papers, and their test accuracy are compared.

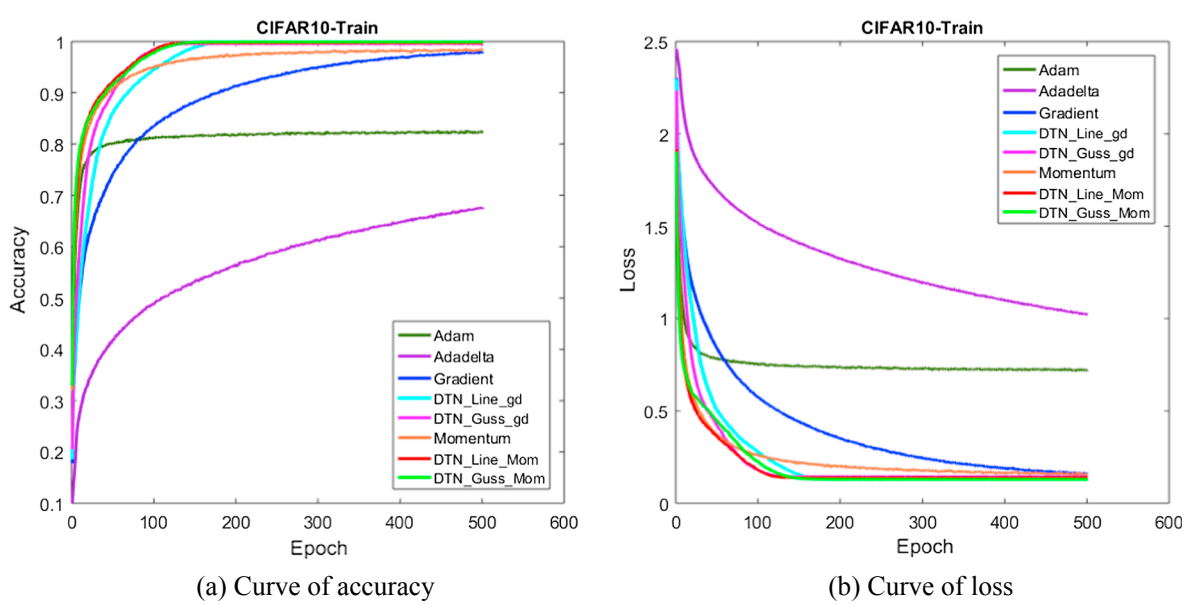
4.2. Performance evaluation on MNIST

MNIST is considered simple and used to solve the problem that involves the classification of digital images [36]. The dataset consists of ten categories with a total of 60k samples. And each sample is a grayscale image of size 28×28. The CNN model used for MNIST is a modern version of LeNet-5. All networks for this dataset were trained for 50k iterations. Two optimization algorithms are used for the training, including gradient descent and Stochastic Gradient Descent (SGD) with momentum value of $\mu = 0.9$. Furthermore, the mini-batch size is 64, and the weights and biases of the network are initialized with zero and no data augmentation is used.

In order to verify the impact of different learning rates on the learning performance of various categories of features in the network, comparison experiments are designed based on the Lenet5 network structure on the MNIST dataset. As shown in Fig. 1(a), the loss function has the largest slope within $[1e-4, 1e-3]$. For balance, experiments on MNIST dataset are performed on the following different learning rates, $\gamma \in \{1e-3, 6e-4, 1e-4\}$. As shown in Fig. 4, the comparison of accuracy between Lenet5 and DTN-Lenet5 on MNIST is illustrated.

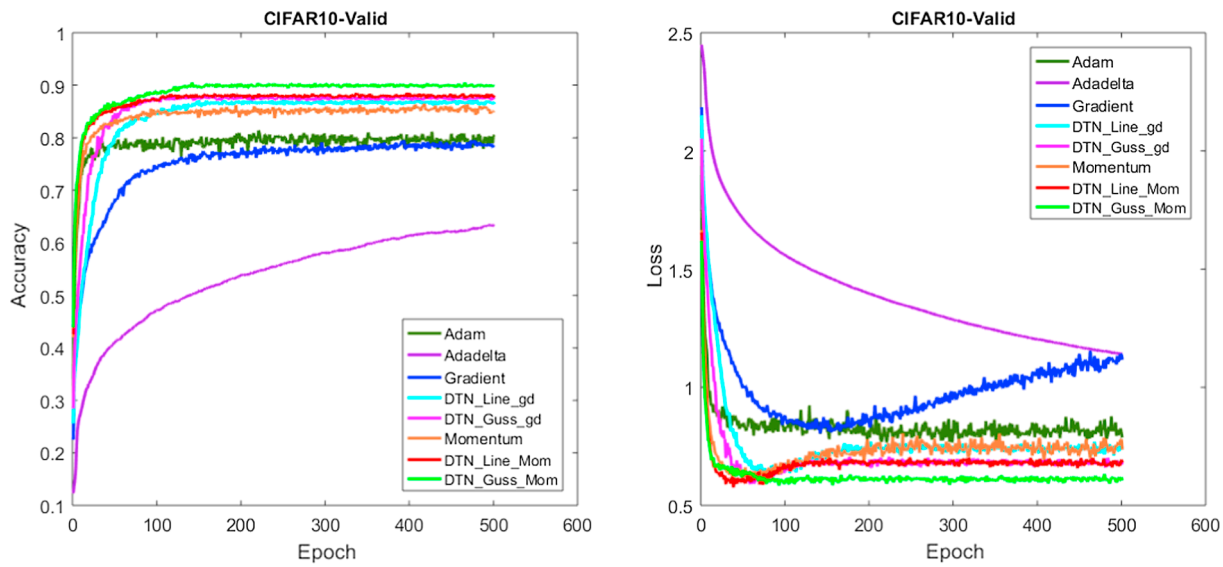
The histogram in Fig. 4 shows various accuracy rates of the testing dataset after 50k training at different learning rates. The accuracy of each class on MNIST testing dataset is evaluated. As shown in Fig. 4(a), when learning rate γ is $1e-3$, the accuracy of almost all categories is relatively higher than other learning rates on MNIST dataset. Averagely, the accuracy at $\gamma = 1e-3$ is the same as that at $\gamma = 6e-4$. The results shown in Fig. 4(a) illustrate that different learning rates have different learning abilities for various features, which affect the overall performance of the classifier. As shown in Fig. 4(b), it is the accuracy of each class of DTN-Lenet5 on MNIST testing dataset. A Gaussian function with a mean of 1 and a standard deviation of 0.25 is used to update the learning rate of each node. The learning rate update policy is a feedback adjustment according to Eq. (6). The T of learning rate update period is 1000 iterations. When learning rate γ is $6e-4$, the accuracy of all categories is relatively higher than other learning rates. The mean accuracy at $\gamma = 6e-4$ is the highest on MNIST testing dataset.

To evaluate the convergence of the proposed DTN framework, the training process of Lenet-5 and DTN is compared by measuring the gradient descent of the loss functions. Lenet-5 is trained with the "start learning rate" of $6e-4$ and the "momentum" of 0.9. The DTN is trained on the MNIST, and learning rate update policy is the feedback adjustment according to Eq. (6). As shown in Fig. 5, it is the curve comparison of different methods on MNIST dataset. The DNT-Lenet5 network framework sets up five learning rate update nodes and a probabilistic fusion node. Further on, the update period of learning rate is set to 1000, and the update threshold is set to 0.01. Here, two methods are used to update the learning rate, which is a uniformly distributed linear function or a Gaussian distributed nonlinear function. In the linear method, the ratio of each node is individually set to 0.8, 0.9, 1.0, 1.1 and 1.2. While in the nonlinear method, the ratio of each node is generated by Gaussian function with a mean of 1 and a standard deviation of 0.25. Fig. 5(a) shows the curve of accuracy obtained by 10k



(a) Curve of accuracy (b) Curve of loss

Fig. 7. Curves of accuracy and loss of different network structures and optimization methods on training dataset.



(a) Curve of accuracy (b) Curve of loss

Fig. 8. Curves of accuracy and loss of different network structures and optimization methods on training dataset.

Table 4
Performance comparisons of ResNet and DTN-ResNet on CIFAR-10 datasets.

ID	Model	Update method	Accuracy
1	ResNet [18]	–	0.8670
2	ResNet	Ada	0.8067
3	ResNet	Adadelta	0.6468
4	ResNet	Gradient	0.8119
5	DTN-ResNet	DTN_Line_gd	**0.9058**
6	DTN-ResNet	DTN_Guss_gd	**0.9128**
7	ResNet	Momentum	0.8689
8	DTN-ResNet	DTN_Line_Mom	**0.9161**
9	DTN-ResNet	DTN_Guss_Mom	**0.9261**

iterations of the training dataset. It shows that DTN-Lenet5 network framework outperforms Lenet5 with higher convergent speed and accuracy. As shown in Fig. 5(b), the loss reduction of DNT framework is more satisfactory than that of Lenet5 with the smallest average loss. After 10k iterations, the loss of Lenet-5 is relatively higher than that of DTN-Lenet5.

As shown in Fig. 6, this is a curve of accuracy generated by top 1000 iterations on MNIST training dataset. The result shows that the convergent speed and accuracy of DTN framework are much higher than that of Lenet-5. Experiments were carried out using Lenet-5 and DTN-Lenet5 as shown in Table 1. The losses of Lenet5 and DNT-Lenet5 keep reducing until 50k iterations. DTN-Lenet5 fuses prediction probabilities from each node in the network into a final prediction. With the same loss function optimization method, DTN-Lenet5 gives better performance than Lenet5. Specifically, DTN-Lenet5 with Gaussian updating learning rate provides notable performance on MNIST testing dataset, which is 0.43% higher than the original Lenet5 algorithm. In the experiment, as documented in Table 2, the performance of the proposed DTN framework is better than all the compared networks. As shown in Table 3, compare with Letnet5, the DTN-Lenet5 improves the accuracy significantly. And with the same iterations, DTN-Lenet5 only takes less than twice the testing time.

4.3. Performance evaluation on CIFAR-10

CIFAR-10 image classification is a more complex problem than MNIST. CIFAR-10 has a total of 60k RGB images of size 32×32 consisting of 10 categories. In order to effectively update the learning rate while training the network, the dataset is divided into 45k samples for the training set, 5k for the verification set and 10k for the testing set. The validation set and the testing set are randomly selected from each category on CIFAR-10.

A topology network structure is proposed to update the learning rate rather than get the most advanced results, so we mainly evaluate the 20-layer ResNet structure. Due to the limitation of data and the increase of training parameters, few CNNs can survive the affect of overfitting. There are two methods to overcome this problem. One is data augmentation, including cropping, flipping and whitening. The other is to adjust the effect of model complexity on the loss function through weight decay. The “weight decay” of 0.0001 and the “momentum” of 0.9 are used, and the weights initialization in [18] and BN [38] are adopted. The training mini-batch of these models is 128 and start with the learning rate of 0.01. The iteration period on the CIFAR-10 dataset is epoch, and the network has experienced 500 epoch iterations.

As shown in Fig. 1(a), the loss function has the largest slope within [0.005, 0.05], so the starting learning rate is set to 0.01. In Figs. 7 and 8, different optimization methods are applied to CIFAR-10 training set and verification set. From the point of comparison, the depth of ResNet is set to 20 layers and do the training work through three optimization methods, including GradientDescentOptimizer, MomentumOptimizer and adamOptimizer. In the DTN-ResNet framework, the depth of each sub-network is 20 layers. Optimization algorithm is gradient descent and SGD with momentum value of $\mu = 0.9$. The updated policy of learning rate is respectively by the linear method and the nonlinear method. Fig. 7 shows the curves of accuracy and loss of different network structures and optimization methods on training dataset.

As shown in Fig. 8(b), benefitting from the feedback adjustment in the proposed loss function, the convergence of DTN is faster than original networks. Besides, the proposed loss function converges faster than the original loss function at the starting and middle stages of the training process (DTN updates learning rate each epoch). Talking about accuracy rate based on training dataset, both ResNet and DTN-ResNet have perfect performance approximating 100% numerically. However, Fig. 8(a) helps select a better one based on verification dataset. As shown in Fig. 8, DTN-ResNet network for this dataset is trained for 500 epochs. Optimization algorithm is a SGD with momentum value of $\mu = 0.9$. The learning rate update policy is a nonlinear method. This model has the highest accuracy on the validation dataset. With consideration

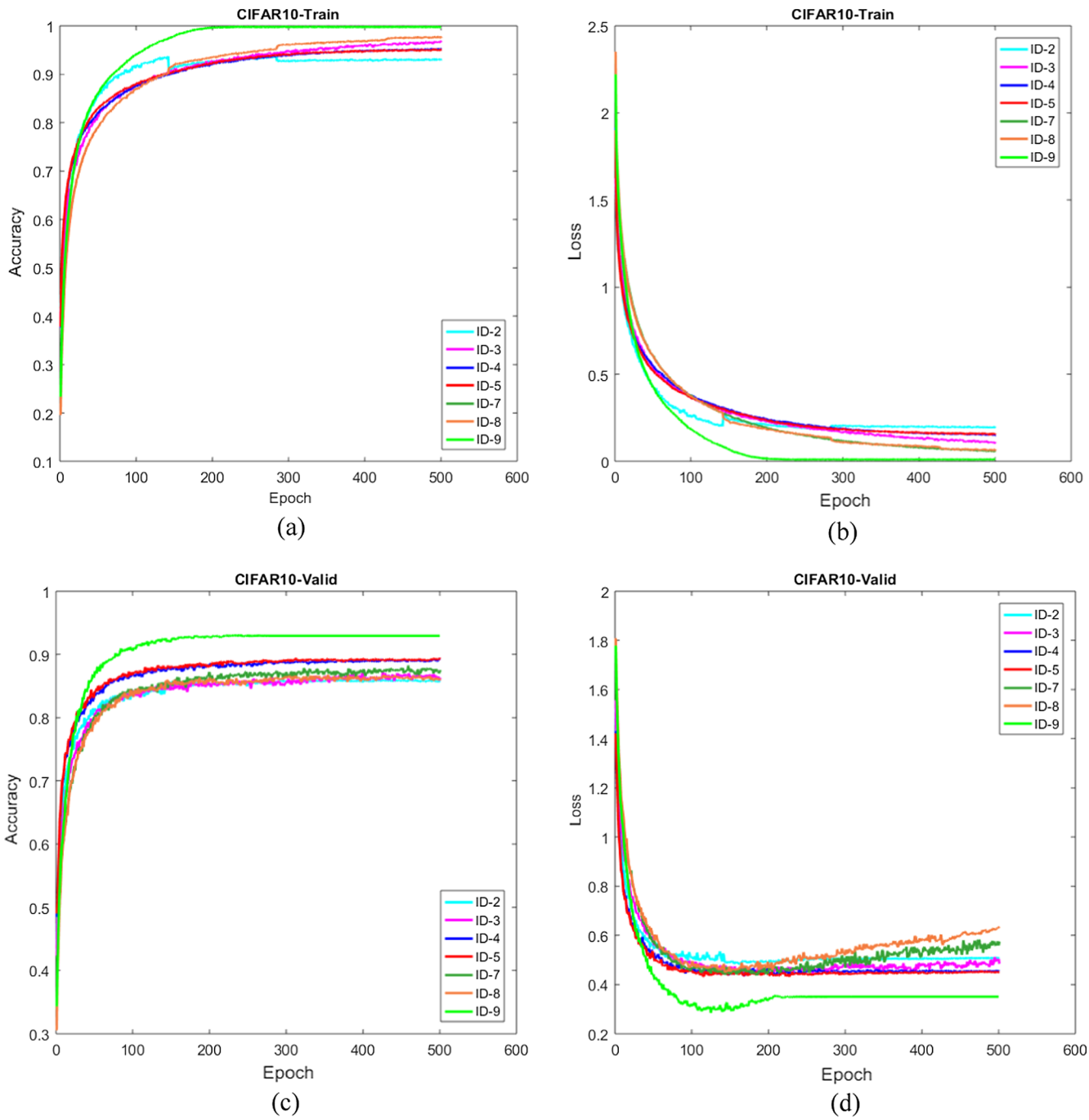


Fig. 9. Performance comparison of Lenet5, DTN-Lenet5, VggNet and DTN-VggNet.

Table 5
The comparison of several network performances on CIFAR-10 datasets.

ID	Model	Update method	Accuracy
	Lenet5 [13]	–	0.6590
	Lenet5	Gradient	0.8307
	Lenet5	Momentum	0.8260
	DTN-Lenet5	DTN_Line_Mom	**0.8568**
	DTN-Lenet5	DTN_Guss_Mom	**0.8579**
	Vgg19 [17]	–	0.9250
	Vgg19	Gradient	0.8397
	Vgg19	Momentun	0.8432
	DTN-Vgg19	DTN_Line_Mom	**0.9082**

and analysis, all results confirm the validity of the proposed theory with respect to the classification (i.e., the fusion probabilities of all nodes), and the topological ResNet network structure shows better capabilities than a single ResNet.

As Table 4 illuminated, the improvement ranges from 3.69% to 11.94%. The accuracy of the DTN framework using nonlinear method and momentum optimization method is 92.16%, 5.72% higher than that of ResNet with Momentum update method.

In order to test the universality of the method, it is applied to two other representative network structures, Lenet5 for dealing with tasks and VggNet19 with medium complexity. As shown in Fig. 9, it is the performance comparison based on CIFAR-10 dataset on applying DNT network and VggNet. Despite a gradually degrading network loss on training set as presented in Fig. 9(b), Fig. 9(d) shows the value of loss

Table 6
The comparison of several network computing time on CIFAR-10 datasets.

ID	Model	Epoch	Training Time(min)	Testing Time(s)	Accuracy
	ResNet	200	80	25	0.8119
	ResNet	400	178	25	0.8331
	DTN-ResNet	200	183	24	**0.9152**
	Lenet5	200	66	17	0.8005
	Lenet5	400	145	20	0.8102
	DTN-Lenet5	200	83	19	**0.8549**
	Vgg19	200	112	15	0.8132
	Vgg19	400	222	40	0.8391
	DTN-Vgg19	200	351	34	**0.9015**

function comes to be stable after 200 epochs on verification set, and even some models have an upward trend. The results state that the generalization ability of the network model has reached a steady state. Fig. 9 reveals the detection rate of different models on the verification set. It can be seen that the implementation of DTN framework based on momentum optimization (i.e. ID-4, ID-5, ID-9) improves accuracy rate compared with Lenet5 and VggNet19.

Particularly, the accuracy of DTN-Lenet5 is significantly better than that of VggNet19. As Table 5 illuminated, in terms of testing dataset, our DTN-Lenet5 (i.e. ID-4, ID-5) exceeds VggNet19 (i.e. ID-7, ID-8) by about 1–2% in accuracy rate and shows more desirable effect in time saving when training and testing data. The results show that the proposed DTN framework has better performance than that of CNN structure. In addition, momentum optimization method applied in linear method based on DTN-Vgg19 has definite advantages whether on a training set or a testing dataset. As Table 5 illuminated, the accuracy of the DTN-VggNet19 exceeds reproduced VggNet19 by about 6–7%. Taking the experimental results above into consideration, it is worth mentioning that the proposed fusing multiple learning rate prediction probabilities in the paper embodies its effectiveness and universality to a great extent.

As Table 6 illuminated, the DTN framework proposed in the paper takes more computing time on training model, but its convergence speed has significant superiority. Additionally, its accuracy is higher than the single network on the test sets. The accuracy of DTN-ResNet (ID-3) is higher (7.61%) than the Vgg19 (ID-8), and costs less computing time on CIFAR-10.

5. Conclusion

In this paper, a DTN framework is proposed based on feedback adjustment learning rate. It provides a universal and powerful scheme to further enhance the performance of the sate-of-the-art DNNs. Meanwhile, many of the involved technical elements, including Topology network structure based on CNN framework, Multiple learning rate feedback iteration and Node prediction probability fusion, collectively contribute to many other pattern recognition related applications in computer vision and other related fields. Moreover, different types of experiments have demonstrated great advantages in terms of accuracy, robustness, convergence, efficiency and versatility of the proposed framework.

However, there are still some limitations in the current DTN framework. According to the above experiments, DTN with multiple learning rate feedback adjustments shows significant performance on MNIST and CIFAR-10 data sets, but is relatively large in terms of computational complexity and training compared with the Lenet and deep residual network. And with the same iterations, DTN framework only takes less than twice the testing time. But the accuracy of the model is significantly improved. The DTN-ResNet training epoch requires only half of the ResNset, but the accuracy of models is higher than ResNet. In the near future, therefore, we will also strive to reduce training time and computational complexity within the proposed framework.

Declaration of Competing Interest

The authors declared that they have no conflicts of interest to this work.

Acknowledgments

This work was supported by the Research on HDR Backlight Liquid Crystal Display Processing Technology Based on Deep Neural Network under Contract HO2018085418.

Appendix A. Supplementary material

Supplementary data to this article can be found online at https://doi.org/10.1016/j.aei.2019.100935.

References

[1] Z. Yousong, Z. Chaoyang, G. Haiyun, W. Jinqiao, Z. Xu, L. Hanqing, Attention couplenet: fully convolutional attention coupling network for object detection, IEEE Trans. Image Process. 1–1 (2018), https://doi.org/10.1109/TIP.2018.2865280.
[2] G. Cheng, P. Zhou, J. Han, Learning rotation-invariant convolutional neural networks for object detection in VHR optical remote sensing images, IEEE Trans. Geosci. Remote Sens. 1–11 (2016), https://doi.org/10.1109/TGRS.2016.2601622.
[3] C. Hisham, J. Jubin, R. Deepu, Backtracking spatial pyramid pooling (spp)-based image classifier for weakly supervised top–down salient object detection, IEEE Trans. Image Process. 1–1 (2018), https://doi.org/10.1109/TIP.2018.2864891.
[4] X. Zhang, Y. Li, Y. Zheng, B. Hou, X. Hou, Joint segmentation and classification of hyperspectral image using meanshift and sparse representation classifier, Geosci. Remote Sens. Sympos. (2014), https://doi.org/10.1109/IGARSS.2013.6723194.
[5] A. Hassan, A. Mahmood, Convolutional recurrent deep learning model for sentence classification, IEEE Access 1–1 (2018), https://doi.org/10.1109/ACCESS.2018.2814818.
[6] C.-Y. Tsai, S.-H. Tsai, Simultaneous 3D object recognition and pose estimation based on RGB-D images, IEEE Access 6 (2018) 28859–28869, https://doi.org/10.1109/ACCESS.2018.2808225.
[7] J. Long, E. Shelhamer, T. Darrell, Fully convolutional networks for semantic segmentation, IEEE Trans. Pattern Anal. Mach. Intell. 39 (4) (2014) 640–651, https://doi.org/10.1109/TPAMI.2016.2572683.
[8] M. Chen, L. Zhang, J.P. Allebach, Learning deep features for image emotion classification, IEEE International Conference on Image Processing, IEEE, 2015. https://doi.org/10.1109/ICIP.2015.7351656.
[9] X. Yu, H. Dong, PTL-CFS based deep convolutional neural network model for remote sensing classification, Computing 100 (8) (2018) 773–785, https://doi.org/10.1007/s00607-018-0609-6.
[10] Y. Ge, S. Jiang, Q. Xu, C. Jiang, F. Ye, Exploiting representations from pre-trained convolutional neural networks for high-resolution remote sensing image retrieval, Multimedia Tools Appl. (2017), https://doi.org/10.1007/s11042-017-5314-5.
[11] H. Wang, J. Jiang, G. Zhang, CraterIDNet: an end-to-end fully convolutional neural network for crater detection and identification in remotely sensed planetary images, Remote Sens. 10 (2018) 1067, https://doi.org/10.3390/rs10071067.
[12] Mateusz Buda, Atsuto Maki, Maciej A. Mazurowski, A systematic study of the class imbalance problem in convolutional neural networks, https://arxiv.org/abs/1710.05381.
[13] Y. Le Cun, Handwritten digit recognition with a back-propagation network, Neural Inform. Process. Syst. (1990).
[14] A. Krizhevsky, I. Sutskever, G.E. Hinton, ImageNet classification with deep convolutional neural networks. Commun. ACM 60, 6 (May 2017), 84–90. https://doi.org/10.1145/3065386.
[15] G.E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever, R. Salakhutdinov, Improving neural networks by preventing co-adaptation of feature detectors. https://arxiv.org/abs/1207.0580.
[16] M.D. Zeiler, R. Fergus, Visualizing and understanding convolutional networks, 2013. https://doi.org/10.1007/978-3-319-10590-1_53.
[17] K. Simonyan, A. Zisserman, Very deep convolutional networks for large-scale image recognition, Comput. Sci. (2014), https://arxiv.org/abs/1409.1556.
[18] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, 2015, https://doi.org/10.1109/CVPR.2016.90.
[19] Y. Zeng, X. Xu, Y. Fang, K. Zhao, Traffic sign recognition using deep convolutional networks and extreme learning machine, in: He X. et al. (Eds.), Intelligence Science and Big Data Engineering. Image and Video Data Engineering. IScIDE 2015. Lecture Notes in Computer Science, 2015, vol. 9242. Springer, Cham. https://doi.org/10.1007/978-3-319-23989-7_28.
[20] Julien Bohné, Y. Ying, Stéphane Gentric, M. Pontil, Learning local metrics from pairwise similarity data, Pattern Recogn. (2017), https://doi.org/10.1016/j.patcog.

2017.04.002 S0031320317301462.
[21] André L. Brun, Alceu S. Britto, Luiz S. Oliveira, Fabricio Enembreck, Robert Sabourin, A framework for dynamic classifier selection oriented by the classification problem difficulty, Patt. Recognit., 76, 175–190. https://doi.org/10.1016/j.patcog.2017.10.038.
[22] C. Boheng, L. Jie, W. Gang, M. Biyun, A novel localized and second order feature coding network for image recognition, Pattern Recogn. 76 (2018) 339–348, https://doi.org/10.1016/j.patcog.2017.10.039.
[23] N.V. Chawla, K.W. Bowyer, L.O. Hall, W.P. Kegelmeyer, Smote: synthetic minority over-sampling technique, J. Artif. Intell. Res. 16 (1) (2002) 321–357, https://doi.org/10.1613/jair.953.
[24] X.Y. Liu, J. Wu, Z.H. Zhou, Exploratory undersampling for class-imbalance learning, IEEE Trans. Cybern. 39 (2) (2009) 539–550, https://doi.org/10.1109/TSMCB.2008.2007853.
[25] S.H. Khan, M. Hayat, M. Bennamoun, F. Sohel, R. Togneri, Cost sensitive learning of deep feature representations from imbalanced data, IEEE Trans. Neural Networks Learn. Syst. 29 (8) (2015) 3573–3587, https://doi.org/10.1109/TNNLS.2017.2732482.
[26] C. Zhang, S. Bengio, M. Hardt, B. Recht, O. Vinyals, Understanding deep learning requires rethinking generalization, , 2016. https://arxiv.org/abs/1611.03530.
[27] Y. Li, N. Wang, J. Shi, X. Hou, J. Liu, Adaptive batch normalization for practical domain adaptation, Pattern Recogn. (2018), https://doi.org/10.1016/j.patcog.2018.03.005 S003132031830092X.
[28] Y. Wang, F. Sun, B. Li, Multiscale neighborhood normalization-based multiple dynamic PCA monitoring method for batch processes with frequent operations, IEEE Trans. Automat. Sci. Eng. 1–12 (2017), https://doi.org/10.1109/tase.2017.2713800.
[29] Mingfei Liu, Wu Wei, Zhenghui Gu, Zhuliang Yu, FeiFei Qi, Yuanqing Li, Deep learning based on batch normalization for P300 signal detection, Neurocomputing 275 (2018) 288–297, https://doi.org/10.1016/j.neucom.2017.08.039.
[30] L.N. Smith, Cyclical Learning Rates for Training Neural Networks, in: 2017 IEEE Winter Conference on Applications of Computer Vision (WACV). IEEE Computer Society, https://doi.org/10.1109/WACV.2017.58.
[31] S. Li, W. Song, H. Qin, A. Hao, Deep variance network: an iterative, improved cnn framework for unbalanced training datasets, Pattern Recogn. (2018), https://doi.org/10.1016/j.patcog.2018.03.035 S0031320318301250.
[32] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, et al., Going deeper with convolutions, 2014. https://doi.org/10.1109/CVPR.2015.7298594.
[33] K. He, X. Zhang, S. Ren, J. Sun, Spatial pyramid pooling in deep convolutional networks for visual recognition, in: D. Fleet, T. Pajdla, B. Schiele, T. Tuytelaars (Eds.), Computer Vision – ECCV 2014. ECCV 2014. Lecture Notes in Computer Science, 2014, vol. 8691. Springer, Cham. https://doi.org/10.1007/978-3-319-10578-9_23.
[34] J. Katarzyna, M.C. Wojciech, On loss functions for deep neural networks in classification, Schedae Informaticae 25 (2016) 9–23, https://doi.org/10.4467/20838476SI.16.004.6185.
[35] Y. LeCun, C. Cortes, C.J. Burges, The MNIST database of handwritten digits, 1998, 2012. Available electronically at < http://yann.lecun.com/exdb/mnist > .
[36] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, Z. Wojna, in: [IEEE 2016 IEEE conference on computer vision and pattern recognition (CVPR) – Las Vegas, NV, USA (2016.6.27-2016.6.30)] 2016 IEEE conference on computer vision and pattern recognition (CVPR) – rethinking the inception architecture for computer vision, 2016, pp. 2818–2826. https://doi.org/10.1109/CVPR.2016.308.
[37] L. Min, C. Qiang, Y. Shuichengm Network in network, 2014. https://arxiv.org/abs/1312.4400.
[38] S. Ioffe, C. Szegedy, Batch normalization: accelerating deep network training by reducing internal covariate shift, 2015. https://arxiv.org/abs/1502.03167.
[39] X. Wu, R. He, Z. Sun, T. Tan, A light cnn for deep face representation with noisy labels, IEEE Trans. Inf. Forensics Secur. 13 (11) (2018) 1–11, https://doi.org/10.1109/TIFS.2018.2833032.